



สำรวจระบบนิเวศ AI Agent Framework

Navigating the AI Agent Framework Ecosystem



เจาะลึกข้อมูลเชิงประจักษ์จาก 1,575 โปรเจกต์ GitHub
และ 11,910 บทสนทนาของนักพัฒนา



รายงานสรุปสำหรับ CTO และ Software Engineers

ยุคทองของ Agent Framework และความสับสนของผู้ใช้งาน

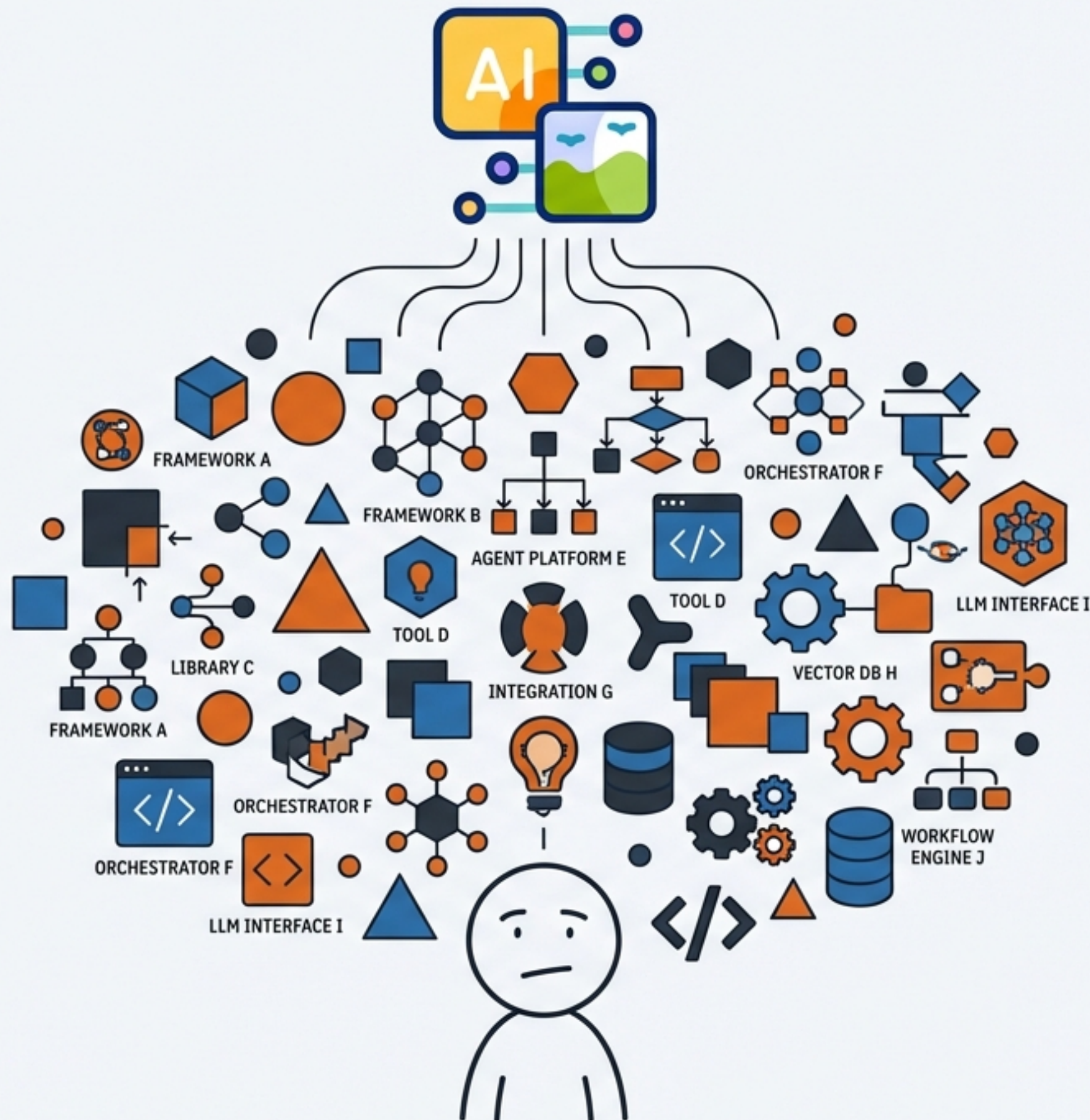
การเติบโตอย่างก้าวกระโดดของ LLM ทำให้เกิดเครื่องมือมากมาย แต่การนำไปใช้จริงและการเลือกเครื่องมือที่เหมาะสมยังเป็นเรื่องท้าทาย

100+ Open-Source Frameworks

มี Framework มากกว่า 100 ตัวบน GitHub ที่มียอดรวมกว่า 400,000 Stars

80%

ของนักพัฒนาพบปัญหาในการเลือกเครื่องมือ



10 ผู้เล่นหลักในโลกของ Agent Development

คัดเลือกจากความนิยม (Popularity) และการใช้งานจริง (Actual Usage) ในโปรเจกต์ GitHub

LangChain 

AutoGen 

LlamaIndex	LangGraph	CrewAI
MetaGPT	Semantic Kernel	BabyACIT
BabyAGI	Swarm	Camel

การแบ่งประเภทตามหน้าที่หลัก



Basic Orchestration (การจัดการพื้นฐาน)

LangChain, LangGraph, Semantic Kernel
เชื่อมต่อ Components และจัดการ Workflow



Multi-Agent Collaboration (การทำงานร่วมกันหลาย Agent)

AutoGen, CrewAI, MetaGPT, Camel
แบ่งงานและแก้ปัญหผ่านการทำงานร่วมกัน



Data Processing (การประมวลผลข้อมูล)

LlamaIndex
เน้น RAG และการเข้าถึงความรู้ภายนอก



Experimental Exploration (การทดลองและสำรวจ)

BabyAGI, Swarm
ทดสอบแนวคิด Autonomous Loop และ
Multi-agent Interface

เทรนด์การใช้งานจริง: Framework เดียวไม่เคยพอ

96% ของโปรเจกต์ชั้นนำใช้ Framework ร่วมกันตั้งแต่ 2 ตัวขึ้นไป

Orchestration + Data

LangChain + LlamaIndex

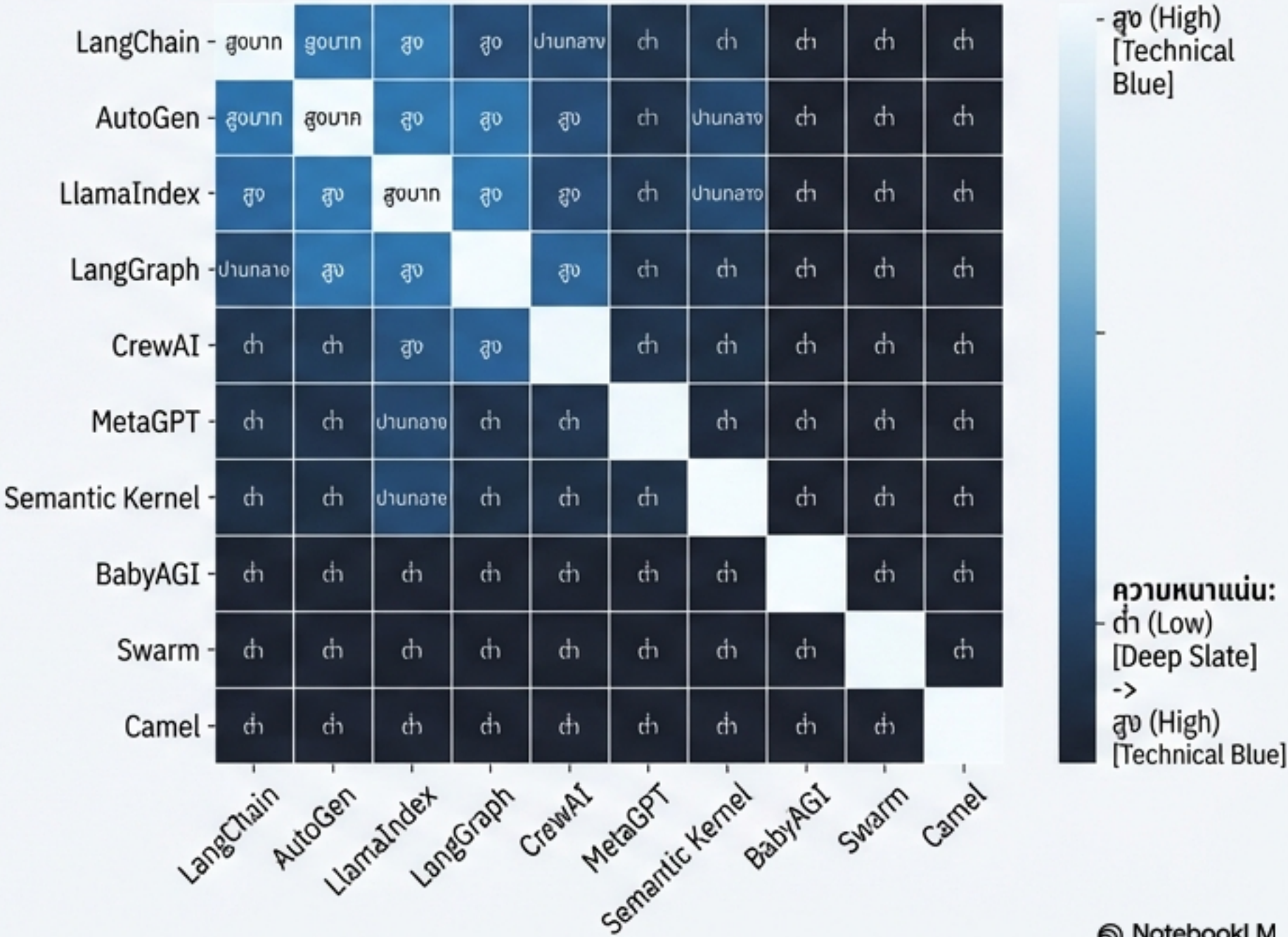
เพื่อการจัดการ Workflow ที่ซับซ้อนพร้อม
ความสามารถ RAG ที่แม่นยำ

Multi-Agent + Orchestration

AutoGen + LangChain

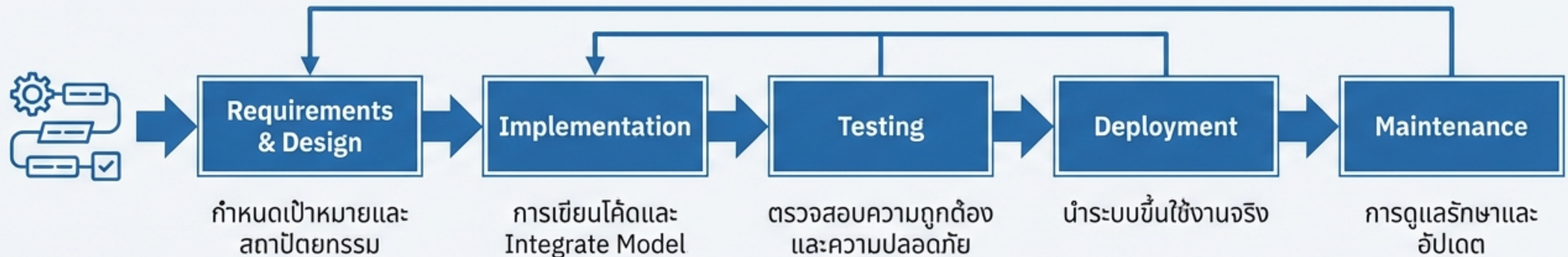
ใช้ AutoGen สำหรับ Agent Conversation
และ LangChain สำหรับการจัดการ Tool

Heat Map: ความถี่ในการใช้งานร่วมกัน (Joint Usage Frequency)



Agent Development คือ Software Engineering

การพัฒนา Agent ต้องผ่านกระบวนการ SDLC (Software Development Life Cycle) เช่นเดียวกับซอฟต์แวร์ทั่วไป



4 อุปสรรคหลักในการพัฒนา

Logic Failures (ความล้มเหลวทางตรรกะ)



ปัญหาจากกลไกการควบคุมภายใน

Tool Failures (ปัญหาเครื่องมือ)



การเชื่อมต่อ API และ Integration

Performance Failures (ประสิทธิภาพ)



การจัดการทรัพยากรและ Context

Version Failures (เวอร์ชัน)



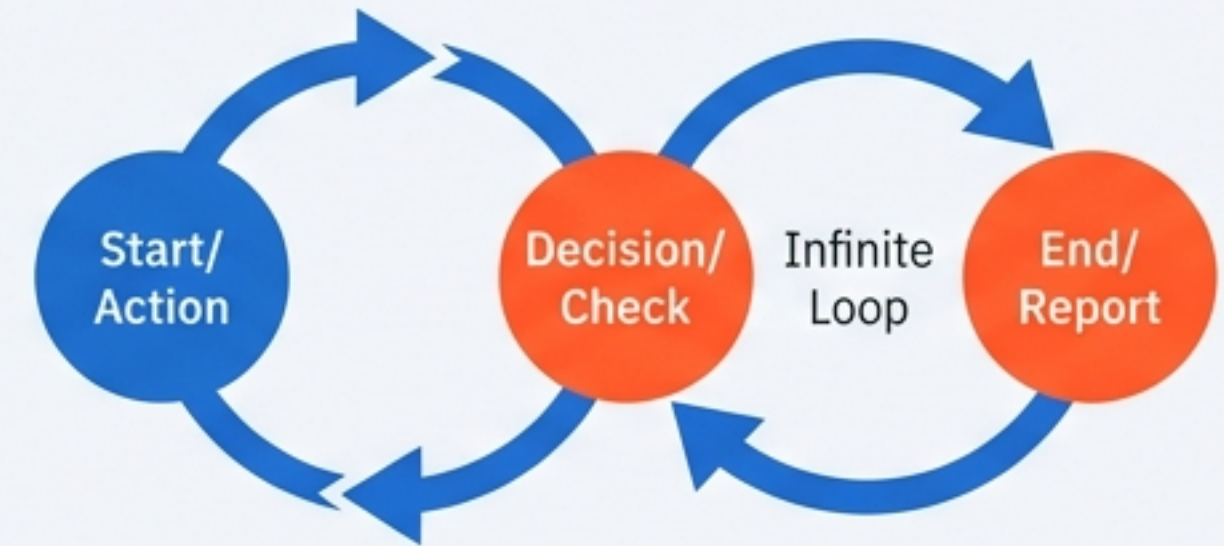
ความไม่เข้ากันจากการอัปเดต

เจาะลึกจุดที่มักล้มเหลว

Logic Issues (25.6% of Logic Failures)

Task Termination Policies (นโยบายการจบงาน)

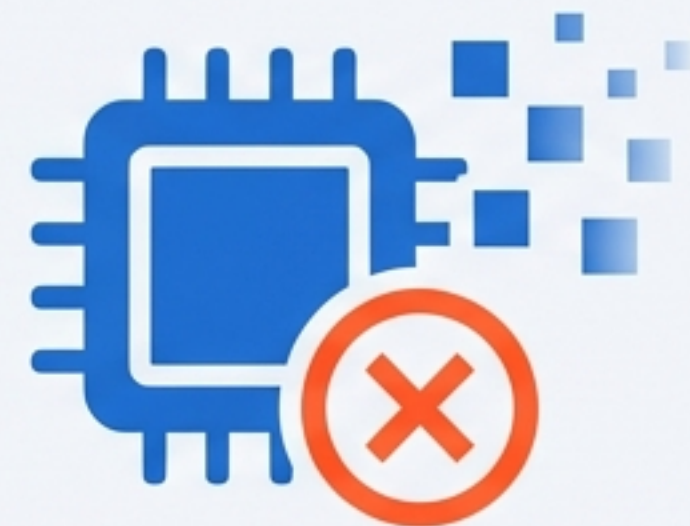
Agent ติดอยู่ใน Infinite Loop หรือไม่สามารถสรุปงานได้



Performance Issues (25% of all challenges)

Context Retention & Memory (การรักษาบริบทและความจำ)

ข้อมูลหายไประหว่างการสนทนา หรือการจัดการหน่วยความจำผิดพลาด



เกณฑ์การวัดผล 5 มิติ



- 1 Development Efficiency**
ความเร็วในการเขียนโค้ดและ Prototyping
- 2 Functional Abstraction**
ความชัดเจนและความง่ายในการใช้งานโมดูล
- 3 Learning Cost**
ความยากง่ายในการเรียนรู้สำหรับผู้เริ่มต้น
- 4 Performance Optimization**
การจัดการทรัพยากรระหว่างรันไทม์
- 5 Maintainability**
ความง่ายในการอัปเดตและขยายระบบ

เริ่มต้นง่าย vs. สร้างงานไว

Best for Beginners
(Learning Cost)

**LangChain &
CrewAI**

ลดกำแพงทางเทคนิคสำหรับผู้เริ่มต้น

Best for Prototyping
(Efficiency)

**AutoGen &
LangChain**

โดดเด่นในการสร้างต้นแบบอย่างรวดเร็ว

ราคาที่ต้องจ่าย: ประสิทธิภาพและการดูแลรักษา



The Universal **Weakness**

Performance Optimization (การปรับแต่งประสิทธิภาพ)

เป็นจุดอ่อนร่วมกันของทุก Framework ยังไม่มีตัวไหนจัดการ Resource ได้สมบูรณ์แบบ

The Cost of Maturity

High Maintenance: LangChain & AutoGen

แม้จะมี Ecosystem ที่แข็งแกร่ง แต่แลกมาด้วยความซับซ้อนในการดูแลรักษา

จุดเด่นเฉพาะตัวที่ไม่ควรมองข้าม



LlamaIndex	LangGraph	MetaGPT
RAG King ยอดเยี่ยมในการจัดการข้อมูลและการดึงความรู้ และการดึงความรู้ (Retrieval-Augmented Generation)	**Complex Logic** แก้ปัญหาข้อจำกัดของ LangChain ด้วย LangChain ด้วยโครงสร้างแบบ Graph (DAG) จัดการ Loop ได้ดีกว่า	**SOP Simulation** จำลองกระบวนการทำงานแบบบริษัทซอฟต์แวร์ แบบบริษัทซอฟต์แวร์ (Standard Operating Procedures) ได้ดีที่สุด

คำแนะนำเชิงกลยุทธ์สำหรับนักพัฒนา



Don't Chase Stars

อย่าตัดสินใจจาก GitHub Stars เพียงอย่างเดียว

Look for Robustness

ให้ความสำคัญกับ **Ecosystem Robustness** (ความแข็งแกร่งของระบบนิเวศ) และ **Long-term Maintenance** (การดูแลรักษาระยะยาว)

Adoption Matters

เลือก Framework ที่มีการใช้งานจริงในโปรเจกต์

สรุปการเลือกเครื่องมือ

Low Barrier to Entry (เริ่มง่าย)	LangChain, CrewAI
Rapid Prototyping (ทำไว)	AutoGen, LangChain
Complex Data/RAG (ข้อมูลซับซ้อน)	LlamaIndex
Complex Workflows (งานซับซ้อน)	LangGraph
Multi-Agent SOPs (ทีมบอท)	MetaGPT, AutoGen

บทสรุป

ระบบนิเวศ Agent กำลังเปลี่ยนแปลง
การผสมผสานเครื่องมือ (Stacking)
และการเตรียมพร้อมรับมือปัญหา
(Logic/Performance) คือกุญแจสู่ความสำเร็จ



Source: An Empirical Study of Agent Developer Practices in AI Agent Frameworks (2025)

Authors: Y. Wang, X. Xu, J. Chen, T. Bi, W. Gu, Z. Zheng.

University: Sun Yat-sen University, Zhejiang University, University of Melbourne, TUM.